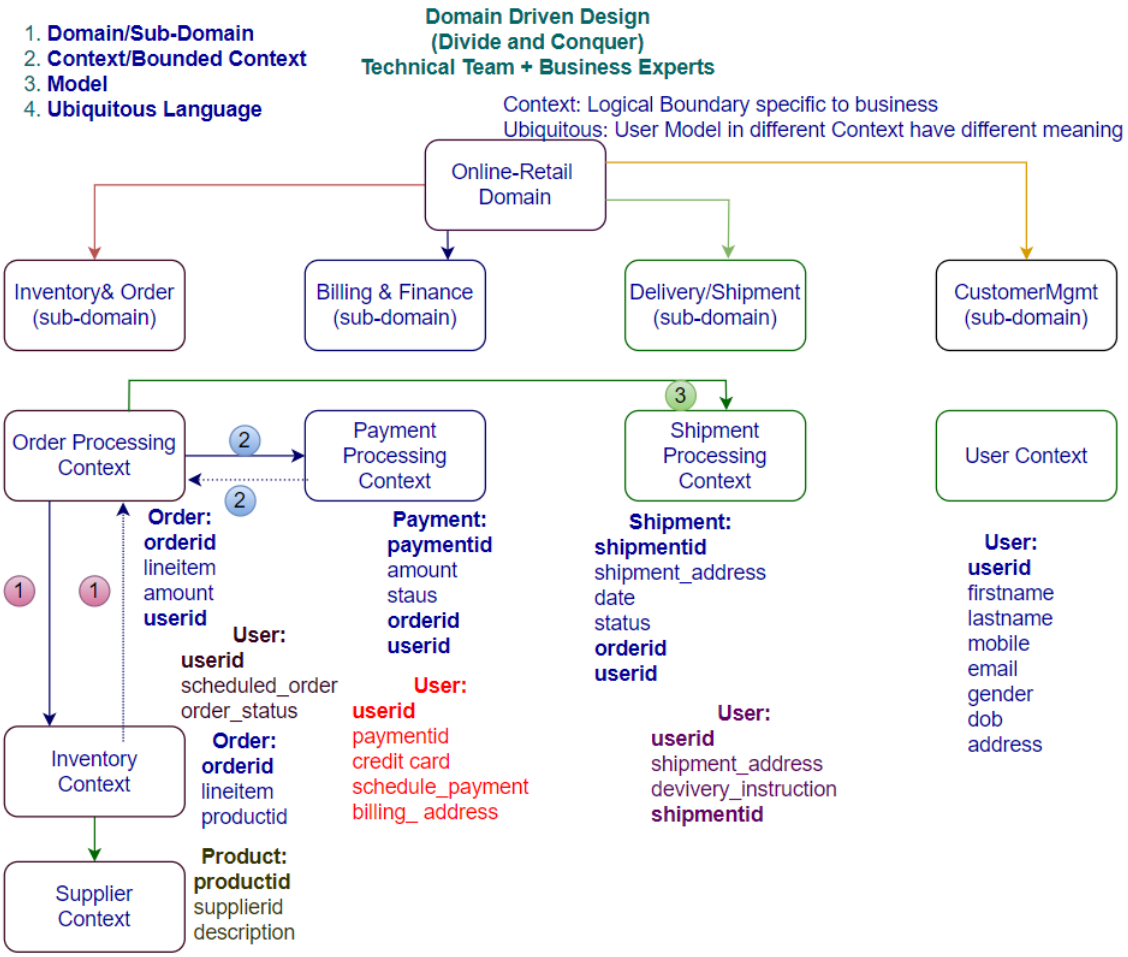
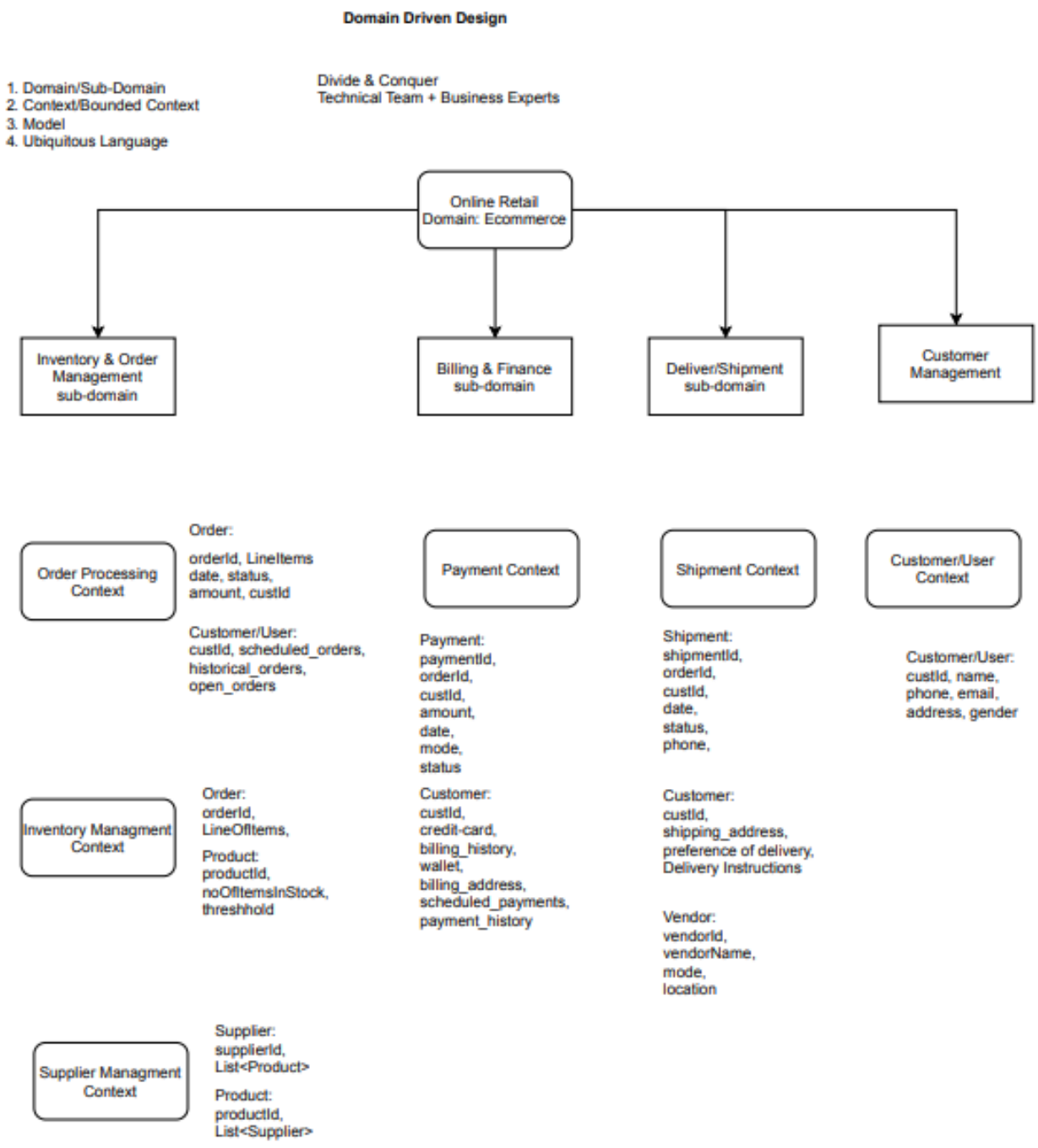
Domain Driven Design

The main objective of DDD is divide and conquer (technical + business experts)

How the organization trends to design their application in the way similar to the work.





There are four aspects of DDD

1. Domain/Subdomain
2. Bounded Context/Context
3. Model (w.r.t context)
4. Ubiquitous Language (applied on the model)

Approach: Divide and Conquer between Technical and Business experts

Bounded Context (logical boundary) specific to the business context, each bounded context maps to the individual ms, this helps us to Identify what ms needs. Logical boundary across the certain functionality of the business.

Each context has their own model, same model (customer)may reside in different contexts, each context has different meaning of the model.

customer in **customers context** as well as customer in **payment context**, and customer in **order processing context**, every context has different meaning of the customer.

In above example **Ubiquitous language (same thing but different meaning in different context)**

UC: Telecom Organization: Telephone

Departments:

- **Customer Context:** Phone calls
- **Operation Context:** A device installed at customer premise.
- **Finance:** Bills
- **Telematics:** A number with an area code

It's a DDD not a database design, DDD is all about the business not about the technology, model may be implemented as database table.

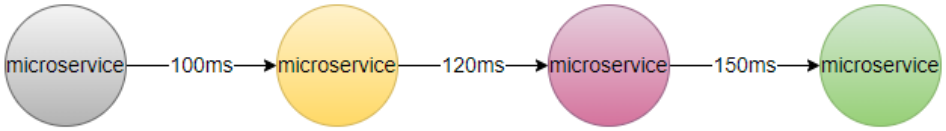
Source of truth: customer Id

Granularity of MS

1) DDD
2) Every request <3-5 ms
3) Benchmarking (recording the performance of application) is always done after code written.

Granularity of Microservices:
1) DDD
2) Thumb rule
3) Benchmark

2) **Thumb Rule:** Every request should not span across more than 3-5 microservice calls.



3) Benchmark (SLA):

GET: /orders, /users 10ms
POST: /orders, /users
PUT/PATCH: /orders/{orderId}, /users/{userId}
DELETE: /orders/{orderId}, /users/{userId}

combine 2-3 ms together (reduce the latenc
decouple 2-3 ms in case of tightly coupled.

Microservice Design Patterns

- **Apigateway** - Zuul/Spring Cloud Gateway/ApiGee, AWS-Apigateway, Nginx
- **Proxy** - Zuul, Nginx
- **Service Discovery/Registry** - Eureka/etc/Zookeeper/Consuul
- **Circuit Breaker** - Hystrix, Resilience4j
- **Client-side load balancing** - Ribbon, SpringCloudLoadBalancer
- **Distributed Tracing** – Sleuth/Zipkin (UI Visualization)
- **Saga** - StateMachine (Orchestrator/Choreography)
- **Metrics Collection** (Actuator Prometheus/Datadog)
- **Strangler Vine** - Migration from Monolith to MSA
- **Shared Data pattern** - Migration from Monolith to MSA
- **Database per service** - MSA

Functional Pattern:

- Aggregator
- Chained
- Branch Pattern
- Event Sourcing
- CQRS

- Log Aggregation (Centralized Logging)
- External Configuration

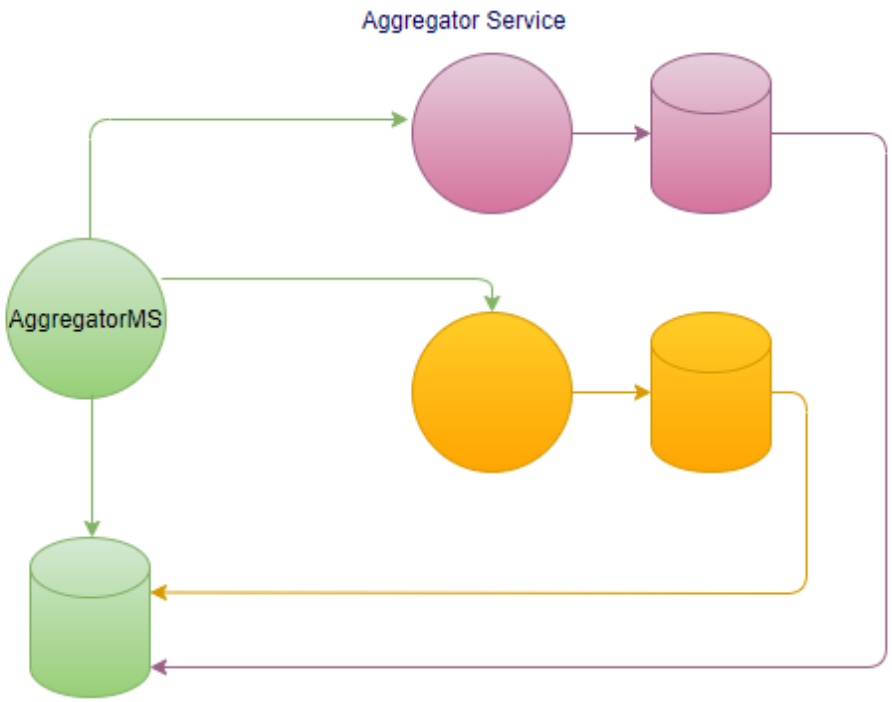
Deployment Design Pattern

- Service per Machine
- Service per Container
- Sidecar

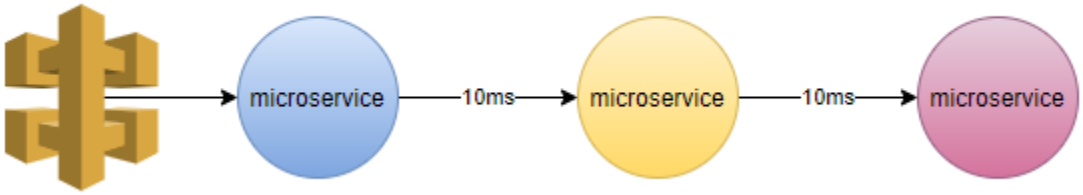
Microservice Design Patterns:

Integration Pattern	Cross Cutting Pattern	Database Pattern	Observability Pattern
1. API Gateway	1. Service Discovery/Registry	1. Database per Service	1. Distributed Tracing
2. Aggregator	2. Circuit Breaker	2. Shared Database	2. Log Aggregation
3. Proxy	3. Bulkhead	3. Saga	3. Metrics
4. Chained	4. External Configuration	4. Event Sourcing	4. Health Check
5. Branch		5. CQRS	

Aggregator Service



Chain Pattern



When go for microservices?

When you want to modular / distributed applications.

Not suitable for low latency applications (go for low level language).

Configuration Management:

1) Properties as a part of code
2) Externalized the properties, at startup
3) Config-Server (Actuator end point /refresh)
4) Config-Server with Messaging Bus (/bus-refresh)

Traditionally way of coding we put configuration file with in the service, properties were part of the application.

rebuild, redeploy, restart

UC: We have 100 ms, each have 5 instances, we have to manage 500 instances.

We must have to externalize the configuration, so that we don't need to do change in the code, because change in code required redeployment, which we need to avoid.

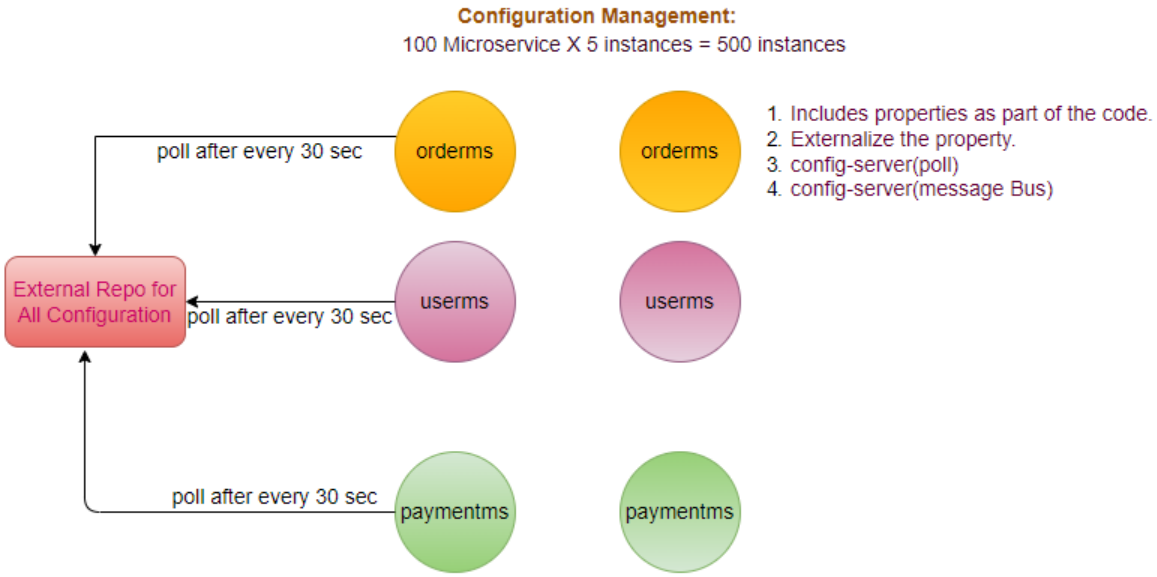
We externalize the configuration, and all instances pull their configuration from the external file on startup.

UC: Once you do the changes in external configuration file, how those changes known to the respective ms?

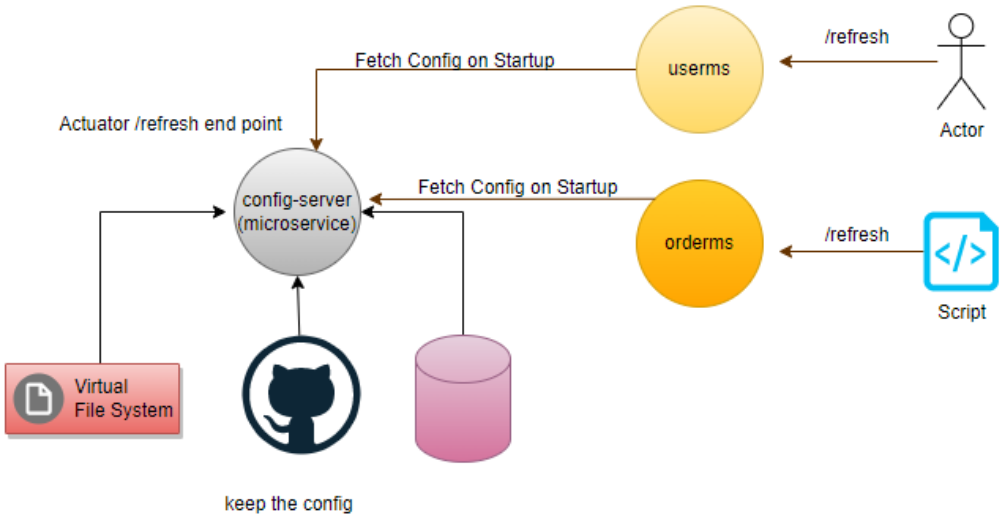
At least, we have to restart the ms, to take latest changes, restart will be manually and will not be optimized solution.

UC: We have to take approach any changes in the external file will be reflected in the respective ms at run time.

solution1: ms poll the changes from the external repository, which will also not an optimized solution because most of the time poll request will be empty, there is few and occasionally changes in the configuration.



solution2: loosely coupled solution (changes at run time), using the config server in that case ms will poll the changes from the config server only when config server notifies them, in that case there will be no any empty poll request will be happened.



UC: We don’t want to restart the sever app, after any change in application. properties, we must have to Externalize the configuration properties, and ensure that there will be Zero downtime.

GitHub/ ConfigServer/ Spring Cloud Bus (Rabbit MQ /Kafka)

step1: user do change in properties file, and push into the GitHub.

step2: As any changes happen on the GitHub, webhook notified the same to the ConfigServer (observer pattern).

How ConfigServer Identified out of 500 instances, changes are for which instance and notified them please refresh your configuration, which is not optimized solution. Rather than notify to each and individual instance we have to notify once and each instance aware that there are some changes in the configuration file (pub-sub model).

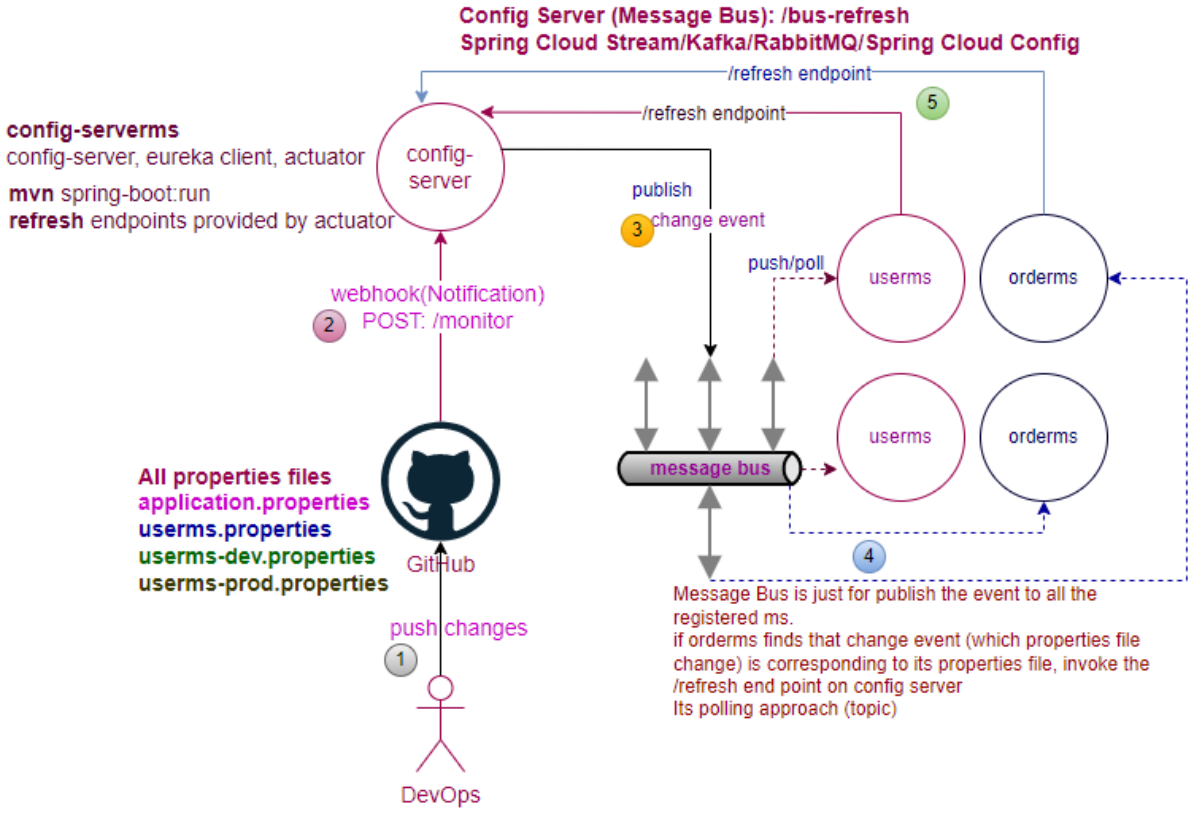
step3: ConfigServer push the changes to Spring Cloud Bus (Kafka/RabbitMQ).

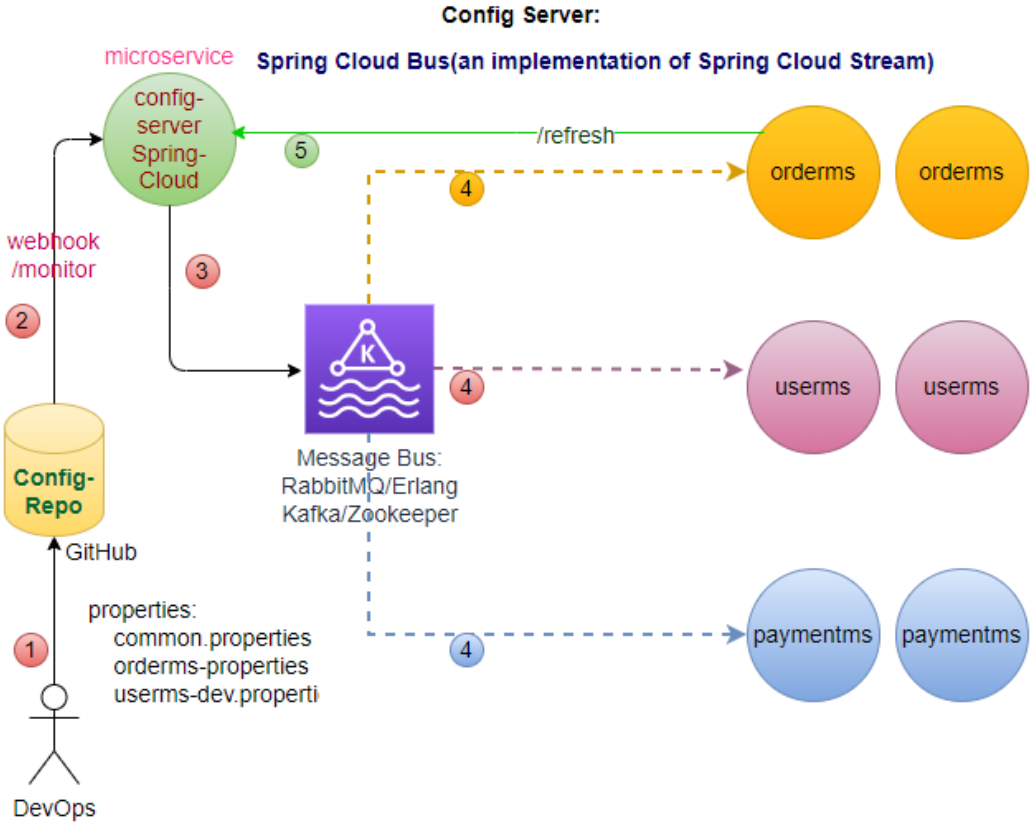
step4: Every ms subscribed to the messaging system

step5: Every ms listens to the Spring Cloud Bus, gets notification (what change).

Step6: Refreshed event is fired.

Step7: As ms knows there is some changes in its property file, will poll the changes from the ConfigServer.


Config Server (Message Bus): /bus-refresh
Spring Cloud Stream/Kafka/RabbitMQ/Spring Cloud Config
/refresh endpoint
/refresh endpoint
5
config-serverms
config-server, eureka client, actuator
mvn spring-boot:run
refresh endpoints provided by actuator
config-server
publish
change event
3
push/poll
userms
orderms
webhook(Notification)
2
POST: /monitor
userms
orderms
message bus
All properties files
application.properties
userms.properties
userms-dev.properties
userms-prod.properties
GitHub
4
Message Bus is just for publish the event to all the registered ms.
if orderms finds that change event (which properties file change) is corresponding to its properties file, invoke the /refresh end point on config server
Its polling approach (topic)
push changes
1
DevOps


Config Server:
microservice
Spring Cloud Bus(an implementation of Spring Cloud Stream)
config-server Spring-Cloud
5
/refresh
orderms
orderms
4
webhook /monitor
3
2
userms
userms
4
Config-Repo
Message Bus: RabbitMQ/Erlang Kafka/Zookeeper
GitHub
properties:
common.properties
orderms-properties
userms-dev.properti
1
paymentms
paymentms
4
DevOps

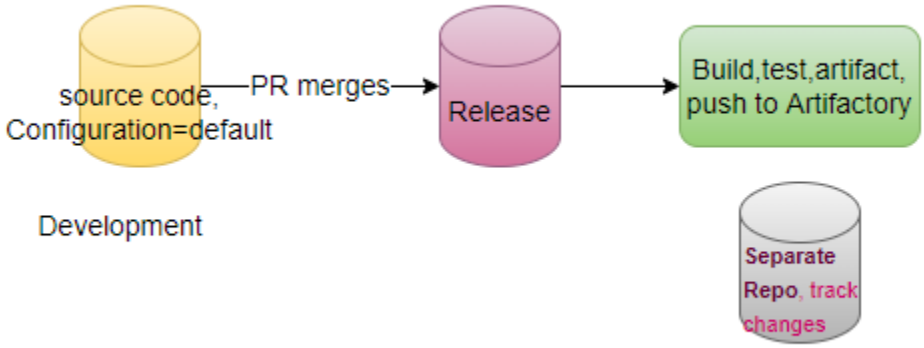
At the time of startup, ms polls the configuration from config-server, if configuration not available only in that case picks the configuration from local properties files.

All ms listens to message bus, message-bus push (RabbitMQ) or ms pull (Kafka). Config change corresponding to the ms, when listen the message will get the message from the config server.

java -jar --spring.profiles.active=dev # activate the profile

Is it event sourcing?

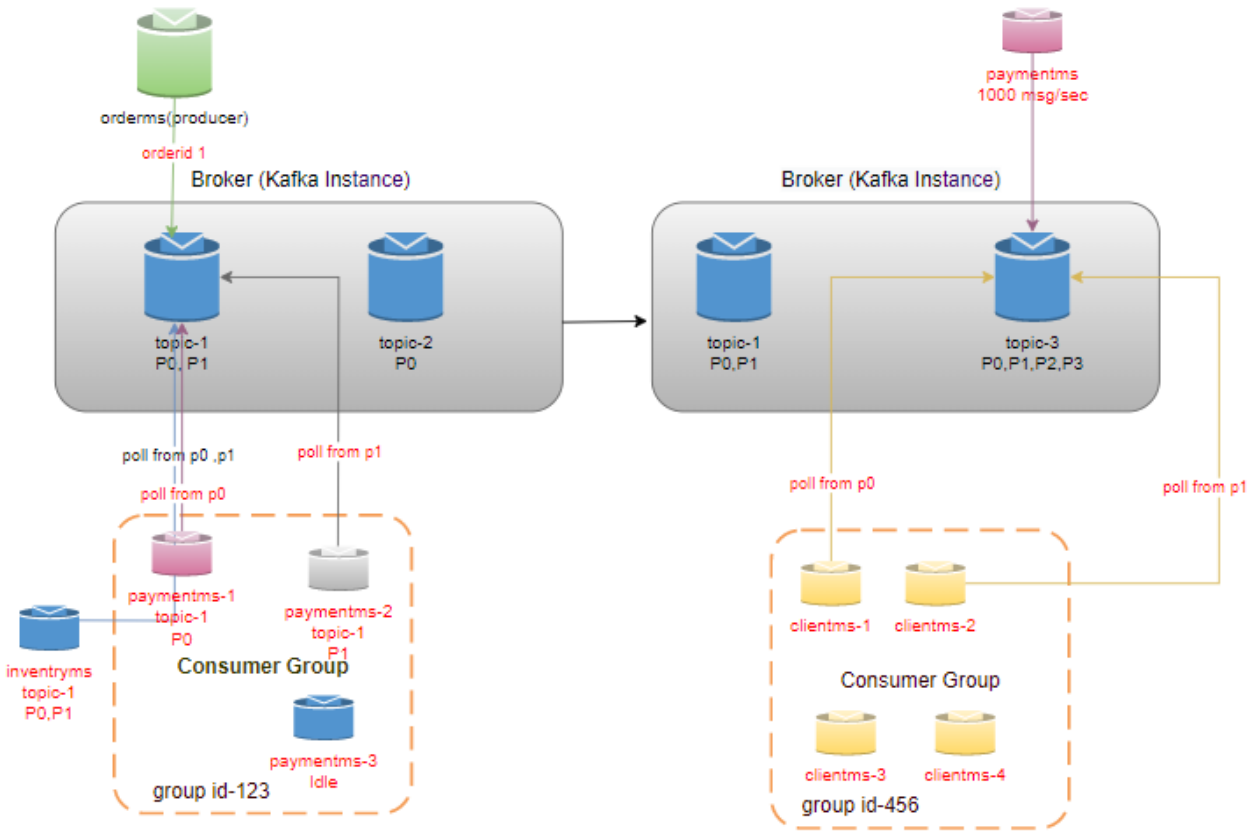
No, in event sourcing we track every event the journey of your data.



Kafka Architecture (Apache Kafka, Confluent Kafka**):** Messaging Model (Kafka-Distributed Messaging System)

1. Broker
2. Topic: Where message send
3. Record (Message)
4. Producer
5. Consumer
6. Consumer Group:(Idempotent group-id)
7. Partition (Scalable): Logically grouping of topic
8. Replication Factor (Highly Available): Number Of Kafka Instance
9. Offset
10. Zookeeper

Kafka:
Apache Kafka
Confluent Kafka



Note: order-1 will be pushed to (P0), order-2 will be pushed to (P1), Internally Kafka do the load balancing at partition level within the topic.

Similarly, within the consumer group payment-1 will be listening to P0 and payment-2 listening to P1, to ensure message processed only once.

Terminology used in Kafka

- Broker: Kafka Instance
- Topic: Each Broker have topics (where message send)

- Partition: Each topic has partitions (logical grouping of topic)- Scalability

  In every partition there is a one master, master is where your writes are happened.

  We can read from any partition.

  How to know where is master (Zookeeper).

- Replication Factor: Number of replicas of partitions (HA/Fault Toleration)
- Record: Messages we produce
- Consumer: Individual service
- Consumer Group: Consumer Can be part of Consumer Group (Only one consumer can take message from the single partition, there is no duplicate message (Idempotent).
- Zookeeper: Is our Orchestrator, know about all the instances. Where our master exists.
  In case of any master failure other instance becomes the master.
- Offset: don't read twice

KAFKA (Poll based): Used for very low latency and high throughput

Writes are not considered successful till written on all partitions. (Based on the configuration).

UC: orderms wants to publish the messages.

step1: messages are published onto the broker (instance of the kafka), may have multiple instances of the broker to provide the HA.

step2: Each broker contains topics (where the messages are published).

step3: Each topics contains multiple partitions (makes kafka scalable).

step4: Each write of record (message) happens only on one master partition, and replicate on the replication partition (HA->Fault Toleration).

step5: consumer consumes the messages e.g., paymentms/inventoryms, may have multiple instances of paymenms all listens to the same topic. By default consumer ms listens to the all the partitions.

step6: consumer polls the messages from the topic, e.g., if orderms publish the message for orderid-1, by default all instances of paymentms polls the message for orderid-1, but we want uniqueness in payment processing.

How we ensure that only 1 instance of paymentms receives the message of orderid-1.

To maintain the uniqueness of processing, we have to implement the consumer group.

step7:Once we implements the consumer group, all the paymentms instances will belong to the 1 consumer group, they will be assign equal distribution of partitions e.g. if there are two instances of paymentms and there are two partitions in the broker then 1-1 partition assign to each individual instance of the paymentms, and if there will be 4 partitions then each one will get 2-2 partitions, and if there will 3 partitions then 1 will get 2 partition and 1 will get 1 partition, if we have 2 partitions and 3 instances in that case 1-1 partition will be assigned to two paymentms instances and 1 instance will be idle, because partition can't be shared, hence we must have either same number of partitions or more number of partitions than the number of instances.

We can add more partitions dynamically but can't reduce the partition dynamically, can't add consumer dynamically (in above case once partition will be added, newly added partition will be assigned to the Idle instance of the paymentms.

UC: In case of broker failure one of the replicated partitions becomes the master and all the master's reference start pointing to the new master with the help of zookeeper.

Zookeeper is for managing the cluster.

Offset

UC: paymentms read 5 records, while inventoryms after reading 3 records gets down, and after those 5 more messages published by the producer, paymentms able to read all 5 messages and read total 10 messages it is updated. inventoryms return back after 5 minutes read only 3 messages before getting down, only 7 remaining messages needs to be read. How will read either from starting or will

read only remaining 7 messages. Logic must be written on consumer side to keep the track offset of 3. Every consumer keeps its own offset, and store the same in kafka. By default, partition keeps messages for 7 days.

UC: We have 5 partitions are available and 5 consumers are available, after some time due to autoscaling scale-down 3 consumers are down.

In that case rebalancing happen, partitions will be distributed among the available consumers.

p1<>c1, p2<>c2, p3<>c3, p4<>c4, p5<>c5

p1,p3,p5<>c1, p2,p5<>c2

Kafka uses topic only

ActiveMQ and RabbitMQ follow the AMQ (Advanced Messaging Queue) protocol (Push based), Kafka has its own protocol. (Poll based).

Kafka used for very high throughput and very low latency.